Data Structure and Algorithm

Generated by Knowledge App | Template: Sage Modern

Data Structure and Algorithm

- Presentation roadmap with 7 core sections
- Progressive coverage from fundamentals to practical understandin
- Focus on concepts, trade-offs, and real-world applicability

What are Data Structures?

- A data structure is a way to organize and store data in a computer
be efficiently accessed and modified.

- Data structures are the building blocks of algorithms and are used
problems in various fields such as computer science, engineering, a

- Examples of data structures include arrays, linked lists, stacks, que
graphs.

Importance of Data Structures

- Efficient use of memory and storage
- Fast data retrieval and manipulation
- Improved algorithm performance
- Better scalability and maintainability

Types of Data Structures

- Arrays: store a collection of elements of the same data type

- Linked Lists: store a sequence of elements, each with a reference to the next element

- Stacks: follow the LIFO (Last In, First Out) principle

- Queues: follow the FIFO (First In, First Out) principle

- Trees: a hierarchical data structure with nodes and edges

- Graphs: a non-linear data structure

Code (python)

```
import array

def create_array():
    arr = array.array('i', [1,
    return arr

def print_array(arr):
    for i in arr:
        print(i)

arr = create_array()
print_array(arr)
```

Array Operations: Insert, Delete, Search

- Insertion: Adding an element at a specific position or at the end of
- Deletion: Removing an element at a specific position or from the e
- Search: Locating an element by its value or index in the array

Time Complexity of Array Operations

- Insertion: O(n) when inserting at a specific position, O(1) when ins
end

- Deletion: O(n) when deleting at a specific position, O(1) when dele
end

- Search: O(n) in the worst case, but can be O(1) with a hash table o

Vector Implementation: Dynamic Arrays

- A vector is a dynamic array that can resize itself when elements are removed
- Vectors use a combination of arrays and linked lists to achieve effic and deletion
- Vectors are commonly used in programming languages to impleme structures like lists and stacks

Linked List Operations

- Insertion: Adding a new node at the beginning or end of the list
- Deletion: Removing a node with a specific value or at a specific pos
- Search: Finding a node with a specific value

Time and Space Complexity of Linked List Opera

- Insertion: O(1) for appending, O(n) for inserting at the beginning
- Deletion: O(1) for deleting the last node, O(n) for deleting a node i
- Search: O(n) in the worst case, O(1) if using a hash table for node s

Linked List Implementation in Python

- Node class: Represents a single node in the list
- LinkedList class: Manages the list and provides operations
- Example usage: Creating a linked list and performing operations

Stack Operations

- Push: add an element to the top of the stack
- Pop: remove an element from the top of the stack
- Peek: view the top element of the stack without removing it
- IsEmpty: check if the stack is empty

Queue Operations

- Enqueue: add an element to the end of the queue
- Dequeue: remove an element from the front of the queue
- Peek: view the front element of the queue without removing it
- IsEmpty: check if the queue is empty

Time and Space Complexity of Stack and Queue

- Push: O(1) time, O(n) space (for dynamic arrays)
- Pop: O(1) time, O(n) space (for dynamic arrays)
- Peek: O(1) time, O(n) space (for dynamic arrays)
- Enqueue: O(1) time, O(n) space (for dynamic arrays)
- Dequeue: O(1) time, O(n) space (for dynamic arrays)

Tree Data Structures

- A tree is a data structure composed of nodes, where each node ha zero or more child nodes.

- Types of trees: B-tree, AVL tree, Binary Search Tree, and more.

- Trees are used for efficient storage and retrieval of data in databas systems.

B-Tree Properties

- A B-tree is a self-balancing search tree data structure that keeps da
allows search, insert, and delete operations in logarithmic time.

- Each node in a B-tree has a fixed number of keys (K) and child nod

- The height of a B-tree is typically much smaller than the height of a
tree with the same number of nodes.

B-Tree Example in Python

- Here's a simple implementation of a B-tree in Python.

- This implementation includes methods for inserting and deleting nodes.

Code (python)

```
class Node:
  def __init__(self, key):
    self.key = key
    self.children = []

class BTree:
  def __init__(self, K):
    self.K = K
    self.root = Node(None)

  def insert(self, key):
    self.root = self._insert(self.roo

  def _insert(self, node, key):
    if node is None:
      return Node(key)

    if len(node.children) == 2 * self
      temp = Node(None)
      node.children.insert(0, temp)
      self._split_child(node, 0)
      return self._insert_child(node,
```

Sorting Algorithms Overview

- Sorting algorithms rearrange elements in a list to satisfy a specific
ascending or descending).

- Common sorting algorithms include Bubble Sort, Selection Sort, In
Merge Sort, and Quick Sort.

- Each algorithm has its strengths and weaknesses, making some mo
specific use cases.

Time Complexity of Sorting Algorithms

- Bubble Sort: O(n^2) in the worst case, making it inefficient for larg

- Selection Sort: O(n^2) in the worst case, similar to Bubble Sort.

- Insertion Sort: O(n^2) in the worst case, but O(n) on average for ne lists.

- Merge Sort and Quick Sort: O(n log n) on average, making them su datasets.

Space Complexity of Sorting Algorithms

- Bubble Sort, Selection Sort, and Insertion Sort: O(1) space complex
only use a constant amount of extra memory.

- Merge Sort and Quick Sort: O(n) space complexity in the worst cas
need to store temporary results.

- In-place sorting algorithms like Insertion Sort can be more memor
small datasets.

Introduction to Searching Algorithms

- Searching algorithms are used to find a specific element in a data s

- They are essential in various applications, including databases, wel engines, and file systems.

- Two popular searching algorithms are Linear Search and Binary Sea

Linear Search

- Linear Search compares each element in the data set with the target element.

- It has a time complexity of O(n), where n is the number of elements in the data set.

- Linear Search is simple to implement but can be slow for large data sets.

Code (python)

```
def linear_search(arr, target):
  for i in range(len(arr)):
    if arr[i] == target:
      return i
  return -1

arr = [3, 5, 2, 7, 9]
target = 5
result = linear_search(arr, tar
print(result)  # Output: 1
```

Binary Search

- Binary Search divides the data set in half and compares the target element with the middle element.

- It has a time complexity of O(log n), making it much faster than Linear Search for large data sets.

- Binary Search requires the data set to be sorted.

Code (python)

```
def binary_search(arr, target):
  low = 0
  high = len(arr) - 1
  while low <= high:
    mid = (low + high) // 2
    if arr[mid] == target:
      return mid
    elif arr[mid] < target:
      low = mid + 1
    else:
      high = mid - 1
  return -1

arr = [1, 2, 3, 4, 5, 6, 7, 8, 9]
target = 5
result = binary_search(arr, target
print(result)  # Output: 4
```

Summary and Next Steps

- Recap of 7 major areas in Data Structure and Algorithm
- Key takeaways to retain for implementation and decision-making
- Suggested path for deeper study and practical application